

```
# Cài đặt các thư viện cần thiết
!pip install -q datasets requests torch peft bitsandbytes transformers trl accelerate senten
```

- Câu lệnh này giúp cài đặt các thư viện quan trọng:
- datasets: Xử lý tập dữ liệu.
- requests: Gửi yêu cầu HTTP.
- torch: Thư viện deep learning của PyTorch.
- peft: Hỗ trợ fine-tuning mô hình bằng LoRA.
- bitsandbytes: Dùng để nén mô hình giúp tiết kiệm bộ nhớ.
- transformers: Thư viện chính để chạy mô hình AI của Hugging Face.
- trl: Hỗ trợ fine-tuning bằng Reinforcement Learning.
- accelerate: Tăng tốc huấn luyện mô hình.
- sentencepiece: Xử lý tokenization cho ngôn ngữ tự nhiên.

```
# Import các thư viện cần thiết
import os
import re
import math
from tqdm import tqdm
from google.colab import userdata
from huggingface_hub import login
import torch
import transformers
from transformers import AutoModelForCausalLM, AutoTokenizer, BitsAndBytesConfig, TrainingArgu
from peft import LoraConfig, PeftModel
from datetime import datetime
```

- Các thư viện này được dùng để:
- os: Quản lý file, thư mục trên hệ thống.
- re: Xử lý chuỗi với biểu thức chính quy.
- math: Thực hiện các phép toán số học.
- tqdm: Hiển thị thanh tiến trình khi chạy vòng lặp.
- google.colab.userdata: Truy cập dữ liệu người dùng trên Google Colab.
- huggingface_hub.login: Đăng nhập vào Hugging Face để tải mô hình.
- torch: Thư viện học sâu của PyTorch.

- transformers: Chạy mô hình AI, xử lý ngôn ngữ tự nhiên.
- AutoModelForCausalLM: Tải mô hình ngôn ngữ tự hồi quy.
- AutoTokenizer: Xử lý tokenization (chuyển văn bản thành token).
- BitsAndBytesConfig: Cấu hình nén mô hình để tiết kiệm bộ nhớ.
- TrainingArguments: Cấu hình tham số huấn luyện.
- set_seed: Đặt seed để đảm bảo kết quả tái lập.
- LoraConfig: Cấu hình fine-tuning LoRA.
- PeftModel: Nạp mô hình đã fine-tune bằng phương pháp PEFT.
- datetime: Làm việc với thời gian, dùng để đặt tên file theo ngày giờ.

```
# Định nghĩa các biến mô hình
BASE_MODEL = "meta-llama/Meta-Llama-3.1-8B"
FINETUNED_MODEL = f"ed-donner/pricer-2024-09-13_13.04.39"
```

- BASE_MODEL: Tên mô hình ngôn ngữ gốc (LLaMA 3.1 với 8 tỷ tham số).
- FINETUNED_MODEL: Đường dẫn đến mô hình đã fine-tune.

```
# Các siêu tham số (hyperparameters) cho QLoRA Fine-Tuning
LORA_R = 32
LORA_ALPHA = 64
TARGET_MODULES = ["q_proj", "v_proj", "k_proj", "o_proj"]
```

- LORA_R = 32: Xác định bậc của ma trận LoRA (số lượng tham số cần học).
- LORA_ALPHA = 64: Hệ số khuếch đại (scaling factor) của LoRA, giúp kiểm soát mức độ ảnh hưởng của các trọng số đã học.
- TARGET_MODULES = ["q_proj", "v_proj", "k_proj", "o_proj"]: Các lớp trọng số của mô hình sẽ được điều chỉnh bằng LoRA.

```
# Đăng nhập vào Hugging Face
hf_token = userdata.get('HF_TOKEN')
login(hf_token, add_to_git_credential=True)
```

- Lấy mã API (HF_TOKEN) của Hugging Face từ dữ liệu người dùng.

- Đăng nhập vào tài khoản Hugging Face để có quyền tải và tải lên mô hình.

```
# Tải mô hình cơ bản không sử dụng nén
base_model = AutoModelForCausalLM.from_pretrained(BASE_MODEL, device_map="auto")

print(f"Memory footprint: {base_model.get_memory_footprint() / 1e9:,.1f} GB")
base_model
```

- AutoModelForCausalLM.from_pretrained(BASE_MODEL, device_map="auto"):
- Tải mô hình LLaMA 3.1-8B từ Hugging Face.
- device_map="auto" giúp tự động xác định thiết bị tốt nhất để tải mô hình (CPU/GPU).
- In ra lượng bộ nhớ mà mô hình chiếm dụng (tính bằng GB).

```
# Tải mô hình với nén 8-bit để tiết kiệm bộ nhớ
quant_config = BitsAndBytesConfig(load_in_8bit=True)

base_model = AutoModelForCausalLM.from_pretrained(
    BASE_MODEL,
    quantization_config=quant_config,
    device_map="auto",
)

print(f"Memory footprint: {base_model.get_memory_footprint() / 1e9:,.1f} GB")
```

- BitsAndBytesConfig(load_in_8bit=True): Kích hoạt nén mô hình xuống 8-bit để giảm dung lượng và tăng tốc độ xử lý.
- Tải lại mô hình với cấu hình nén 8-bit.
- In ra dung lượng bộ nhớ sau khi nén.

```
# Tải tokenizer và mô hình với nén 4-bit
quant_config = BitsAndBytesConfig(
    load_in_4bit=True,
    bnb_4bit_use_double_quant=True,
    bnb_4bit_compute_dtype=torch.bfloat16,
    bnb_4bit_quant_type="nf4"
)

base_model = AutoModelForCausalLM.from_pretrained(
    BASE_MODEL,
    quantization_config=quant_config,
    device_map="auto",
)

print(f"Memory footprint: {base_model.get_memory_footprint() / 1e9:,.2f} GB")
```

- Giảm kích thước mô hình xuống 4-bit với cấu hình:
- `load_in_4bit=True`: Sử dụng nén 4-bit.
- `bnb_4bit_use_double_quant=True`: Áp dụng double quantization để tiết kiệm bộ nhớ hơn.
- `bnb_4bit_compute_dtype=torch.bfloat16`: Sử dụng định dạng số bfloat16 để tính toán chính xác hơn.
- `bnb_4bit_quant_type="nf4"`: Sử dụng phương pháp nén NF4 tối ưu hóa hiệu suất.
- Tải mô hình với cấu hình này và hiển thị dung lượng bộ nhớ sau khi nén.

```
# Tải mô hình fine-tuned từ Hugging Face
fine_tuned_model = PeftModel.from_pretrained(base_model, FINETUNED_MODEL)

print(f"Memory footprint: {fine_tuned_model.get_memory_footprint() / 1e9:,.2f} GB")

fine_tuned_model
```

- `PeftModel.from_pretrained(base_model, FINETUNED_MODEL)`:
- Tải mô hình đã fine-tune dựa trên `base_model`.
- Hiển thị bộ nhớ mà mô hình fine-tuned chiếm dụng.

```

# Tính toán số lượng tham số của các lớp LoRA
lora_q_proj = 4096 * 32 + 4096 * 32
lora_k_proj = 4096 * 32 + 1024 * 32
lora_v_proj = 4096 * 32 + 1024 * 32
lora_o_proj = 4096 * 32 + 4096 * 32

# Tổng số tham số của một lớp
lora_layer = lora_q_proj + lora_k_proj + lora_v_proj + lora_o_proj

# Mô hình có 32 lớp
params = lora_layer * 32

# Tính kích thước mô hình (MB)
size = (params * 4) / 1_000_000

print(f"Total number of params: {params:,} and size {size:,.1f}MB")

```

- Tính toán số lượng tham số của các lớp LoRA trong mô hình:
- `lora_q_proj`, `lora_k_proj`, `lora_v_proj`, `lora_o_proj`: Các ma trận LoRA cho từng lớp của Transformer.
- Mỗi lớp có `lora_layer` tham số.
- Mô hình có 32 lớp, tổng số tham số là `params`.
- Kích thước của mô hình được tính bằng MB.
- In ra tổng số tham số và kích thước của mô hình sau khi áp dụng LoRA.